

Lịch sử Tech Stack Decisions — Lishop

Tài liệu này ghi lại quá trình research, các lựa chọn được cân nhắc, và quyết định cuối cùng cho từng thành phần trong hệ thống Lishop.

1. Monorepo Tool

Tiêu chí	pnpm Workspaces	Turborepo	Nx	Lerna
Speed	Trung bình	Nhanh (caching)	Nhanh	Chậm
Configuration	Minimal	Đơn giản	Phức tạp	Trung bình
Remote caching	✗	✓ Vercel Remote Cache	✓ Nx Cloud	✗
Task orchestration	✗	✓	✓	✓
Learning curve	Thấp	Thấp	Cao	Thấp

Quyết định: Turborepo + pnpm Workspaces

- **pnpm** làm package manager vì `strict` mode giúp phát hiện dependency missing sớm, tiết kiệm disk space với content-addressable storage.
- **Turborepo** làm task orchestrator vì cấu hình đơn giản (turbo.json), có sẵn remote caching, zero runtime overhead.

Alternatives bị loại:

- Lerna: Quá chậm, không còn được 维护 tích cực so với Turborepo.
- Nx: Quá phức tạp cho monorepo với ~15 packages, cần học plugin system riêng.
- pnpm workspaces alone: Không có task orchestration, mỗi lần build phải chạy script thủ công.

2. Frontend Framework

Tiêu chí	Next.js 15	Remix	Vite + React SPA	Gatsby
SSR/SSG	✓ App Router	✓	✗ (cần thêm)	✓
Module Federation	✓ Official plugin	✗	✓	✗
i18n	✓ next-intl	✓	Thủ công	✗
Image optimization	✓ built-in	✗	✗	✓
Server Components	✓ RSC	✗	✗	✗
Community size	Lớn nhất	Trung bình	Lớn	Suy giảm

Quyết định: Next.js 15 (App Router)

- App Router với React Server Components giúp giảm bundle size cho các trang public (product listing, landing).
- `next/image`, `next/dynamic`, `generateMetadata` có sẵn, không cần thêm thư viện.
- Module Federation plugin từ community hoạt động ổn định, phù hợp với kiến trúc micro-frontend.

Alternatives bị loại:

- Remix: Không hỗ trợ Module Federation, không có image optimization built-in, cần nhiều custom setup.
- Vite + React SPA: Không SSR, SEO kém, phải tự setup routing, không phù hợp e-commerce platform.
- Gatsby: Không phù hợp với dynamic content (e-commerce), build time chậm với catalog lớn.

3. Micro-Frontend Strategy

Tiêu chí	Module Federation	Single SPA	iFrames	qiankun
Shared dependencies	✅ native	✅ SystemJS	❌	✅
Performance	Tốt	Trung bình	Kém	Trung bình
Router integration	✅ seamless	✅	❌	✅
Bundle size	Tối ưu (shared)	Trung bình	Mỗi app full bundle	Trung bình
Next.js support	✅	❌	✅	❌
Dev experience	Tốt (độc lập)	Phức tạp	Tốt	Trung bình

Quyết định: Module Federation (`@module-federation/nextjs-mf v8`)

- Cho phép mỗi MFE deploy độc lập, share dependencies phổ biến (React, zustand, axios) để giảm bundle size.
- Tích hợp liền mạch với Next.js App Router.
- Shell app quản lý layout chung (header, footer), mỗi MFE là một autonomous app riêng.

Cấu trúc MFEs:

- 11 MFEs: auth, catalog, cart, checkout, orders, profile, promotions, notifications, admin, seller, shop-chat
- 1 Shell app: layout + event bus
- Giao tiếp cross-MFE qua `BroadcastChannel` API (event-bus package)

Alternatives bị loại:

- Single SPA: Cần SystemJS, loading chậm, routing phức tạp với Next.js.
- iFrames: Performance kém, khó quản lý state cross-frame, không responsive tốt.
- qiankun: Thiên về SPA, không hỗ trợ Next.js App Router.

4. Backend Framework & HTTP Server

Tiêu chí	NestJS + Fastify	NestJS + Express	Fastify (pure)	Express (pure)
Performance	❤️ Cao	Trung bình	❤️ Cao	Thấp
Validation (DTO)	✅ class-validator	✅	❌	❌
Swagger/OpenAPI	✅ @nestjs/swagger	✅	❌ (thêm)	❌
Module system	✅	✅	❌	❌
WebSocket	✅ @nestjs/websockets	✅	Thủ công	Thủ công
Rate limiting	✅ @nestjs/throttler	✅	Thủ công	Thủ công
Ecosystem	Lớn	Lớn	Nhỏ	Lớn
Request/sec (hello world)	~45K	~13K	~50K	~8K

Quyết định: NestJS 10 + Fastify (`@nestjs/platform-fastify`)

- Fastify nhanh hơn Express ~3-4x, quan trọng khi xử lý realtime (Socket.IO) + API calls đồng thời.
- NestJS mang lại structure chuẩn (modules, services, controllers, guards, interceptors), validation tự động.
- `@nestjs/swagger` tự động generate OpenAPI spec từ decorators.

Alternatives bị loại:

- Express: Chậm, không có structure chuẩn, phải tự setup mọi thứ (validation, swagger, rate limiting).
- Fastify pure: Thiếu module system, không có dependency injection, code khó maintain với 26 modules.
- NestJS + Express: Dễ migrate hơn nhưng hiệu năng kém hơn, không tận dụng được Fastify hooks.

5. Database & ORM

Tiêu chí	PostgreSQL + Prisma	PostgreSQL + TypeORM	PostgreSQL + Drizzle	MongoDB + Mongoose
Type safety	✅ Xuất sắc	Trung bình	✅ Tốt	Kém
Migration	✅ Prisma Migrate	✅	✅	Thủ công
Query performance	Trung bình (layer)	Tốt	❤️ Tốt nhất	Tốt
Relations	✅	✅	✅	❌ (NoSQL)
Admin UI	✅ Prisma Studio	❌	❌	✅ Compass
Transactions	✅	✅	✅	Trung bình
Ecosystem	Lớn	Lớn	Đang phát triển	Lớn

Quyết định: PostgreSQL 16 + Prisma 5

- Prisma type safety xuất sắc — schema → type tự động, IDE autocomplete đầy đủ.
- Prisma Migrate quản lý migration tập trung, dễ review.

- Prisma Studio cho phép debug data trực tiếp.
- PostgreSQL là lựa chọn an toàn cho e-commerce (ACID, transactions, relations phức tạp).

Alternatives bị loại:

- TypeORM: Query builder kém type safety, relations dễ lỗi, migration phức tạp hơn. Không có Prisma Studio tương đương.
- Drizzle: Query builder nhanh hơn Prisma, nhưng migration mới hơn và chưa stable, thiếu admin UI, dev tool ít hơn.
- MongoDB: NoSQL không phù hợp với e-commerce có nhiều relations (users ↔ orders ↔ products ↔ payments ↔ addresses).

6. State Management (Frontend)

Tiêu chí	TanStack Query + Zustand	Redux Toolkit	Redux Toolkit + RTK Query	Jotai	Valtio
Server state	✔ Xuất sắc	✘ (tự làm)	✔ RTK Query	✘	✘
Client state	✔ Zustand	✔	✔	✔	✔
Boilerplate	Thấp	Cao	Trung bình	Thấp	Thấp
DevTools	✔ React Query Devtools	✔ Redux DevTools	✔	✘	✘
Bundle size	~15KB	~12KB (core)	~18KB	~7KB	~6KB
Cross-MFE state	✔ BroadcastChannel	✘	✘	✘	✘

Quyết định: TanStack React Query 5 + Zustand 4

- **TanStack React Query** cho server state: caching, refetch, optimistic updates, pagination support, mutation với rollback.
- **Zustand** cho client state: Auth store, UI state (theme, sidebar), global modals.
- Cross-MFE event bus riêng (BroadcastChannel) thay vì share store giữa các MFEs.

Alternatives bị loại:

- Redux Toolkit: Boilerplate nhiều hơn, không có caching built-in cho API calls (phải dùng RTK Query). Không cần thiết với scope project này.
- Jotai/Valtio: Quá mới, không có dev tools mạnh, ít tài liệu production.

7. UI Framework & Styling

Tiêu chí	Tailwind CSS + Radix UI	shadcn/ui	Material UI	Ant Design	Chakra UI
Bundle size	❤️ Nhỏ	Nhỏ	❤️ Lớn (~200KB)	Lớn (~300KB)	Trung bình
Customizability	❤️ Tối đa	Cao	Thấp	Thấp	Cao
Headless primitives	✅ Radix	✅ Radix	❌	❌	❌
Design tokens	✅ tailwind.config	✅ CSS vars	❌	❌	✅
Vietnamese support	✅ (manual)	✅ (manual)	✅	✅	✅
Server Components	✅	✅	❌	❌	❌

Quyết định: Tailwind CSS 4 + Radix UI + shadcn/ui patterns

- **Tailwind CSS:** Utility-first, bundle nhỏ (purge), build nhanh, dễ tạo design system với design tokens.
- **Radix UI:** Headless, accessible, unstyled — kiểm soát hoàn toàn UI qua Tailwind.
- **shadcn/ui patterns:** Component structure (CVA + cn() + `asChild` pattern), code ownership (không phải dependency).

Alternatives bị loại:

- Material UI: Bundle quá lớn, khó custom ra khỏi Material Design guidelines.
- Ant Design: Quá nặng, khó custom, không phù hợp với Tailwind.
- Chakra UI: Phụ thuộc nhiều vào styled-components/emotion, performance kém hơn Tailwind.

8. Payment Gateways

Tiêu chí	Stripe	PayPal	VNPAY	MoMo	ZaloPay
Vietnamese market	❌ (quốc tế)	❌	✅	✅	✅
Fees	2.9% + \$0.30	4.4% + \$0.30	0.8%-2%	1.5%-2.5%	1.5%-2.2%
Setup complexity	Thấp	Thấp	Cao (cần merchant)	Trung bình	Trung bình
Webhook support	✅	✅	✅ (IPN)	✅	✅
Test environment	✅	✅	✅	✅	✅
Refund API	✅	✅	✅	✅	✅
Recurring	✅	✅	❌	✅	❌

Quyết định: Tích hợp tất cả 5 gateways

- **Stripe, PayPal:** Cho người dùng quốc tế, thanh toán bằng USD/Card.
- **VNPAY, MoMo, ZaloPay:** Cho thị trường Việt Nam (QR code, ATM, ví điện tử).
- Mỗi gateway implement qua strategy pattern — để thêm/bớt mà không ảnh hưởng đến payment flow chính.
- Payment processing queue qua BullMQ để xử lý async (tránh timeout với VNPAY).

9. Search Engine

Tiêu chí	MeiliSearch	Elasticsearch	Algolia	PostgreSQL full-text
Self-hosted	✓	✓	✗	✓ (built-in)
Speed	♥ Dưới 50ms	100-200ms	Dưới 50ms	200-500ms
Vietnamese support	✓ (tokenizer)	✗ (cần plugin)	✓	✗
Typo tolerance	✓	✓	✓	✗
Faceted search	✓	✓	✓	✗
Setup complexity	♥ Thấp (Docker)	Cao	Thấp (SaaS)	Thấp
Cost	Free (self-hosted)	Free (self-hosted)	♥ \$1.50/1K records	Free

Quyết định: MeiliSearch

- Dễ setup nhất trong các self-hosted search engines (Docker image sẵn).
- Vietnamese tokenizer hỗ trợ tốt (tách từ tiếng Việt, tìm kiếm dấu/không dấu).
- Typo tolerance tích hợp sẵn, không cần cấu hình phức tạp.
- REST API đơn giản, documents indexing dễ dàng.

Alternatives bị loại:

- Elasticsearch: Setup phức tạp, memory-heavy, cần plugin ICU cho Vietnamese tokenizer.
- Algolia: Chi phí cao (SaaS), không kiểm soát được data, không phù hợp startup.
- PostgreSQL full-text: Không hỗ trợ Vietnamese, không có typo tolerance, không faceted search.

10. Caching & Message Queue

Tiêu chí	Redis	BullMQ	RabbitMQ	Kafka
Caching	✓ Primary	✗	✗	✗
Queue	✗	✓	✓	✓
Pub/Sub	✓	✓	✓	✓
Persistence	✓ RDB/AOF	✓ (Redis)	✓	✓
Performance	♥ Microsecond	♥ (Redis)	Milliseconds	♥ High throughput
Setup	♥ Docker	✓ (trên Redis)	Trung bình	Cao

Quyết định: Redis 7 + BullMQ

- Redis đáp ứng cả caching (token blacklist, session cache, rate limiting counters) và job queue qua BullMQ.
- BullMQ tận dụng Redis persistence, hỗ trợ delayed jobs, repeatable jobs, job scheduling.
- Chỉ cần chạy một service Docker (Redis) thay vì hai.

Alternatives bị loại:

- RabbitMQ: Cần chạy thêm service riêng, không dùng được cho caching. Overkill với nhu cầu queue hiện tại (chủ yếu email queue).
- Kafka: Overkill hoàn toàn, phù hợp với event streaming data lớn, không cần thiết cho e-commerce platform vừa.

11. Real-time Communication

Tiêu chí	Socket.IO	WebSocket (native)	Server-Sent Events	Polling
Bidirectional	✓	✓	✗ (server → client)	✓ (giả lập)
Fallback transport	✓ (long-polling)	✗	✓	✓
Room support	✓	✗ (tự làm)	✗	✗
Reconnection	✓ auto	✗ (tự làm)	✓ tự động	✗
NestJS integration	✓ @nestjs/websockets	✓	✓	✗
Browser support	✓ rộng	✓ (>=2015)	✓	✓ mọi nơi

Quyết định: Socket.IO 4

- Auto reconnection, fallback transport (khi WebSocket bị chặn), room-based messaging là những tính năng quan trọng không muốn tự implement.
- NestJS có `@nestjs/websockets` + `@nestjs/platform-socket.io` tích hợp sẵn.
- Dùng cho: chat shop ↔ customer, realtime notifications, order status updates.

Alternatives bị loại:

- Native WebSocket: Phải tự implement reconnection, heartbeat, room management — quá nhiều boilerplate.
- SSE: Chỉ một chiều (server → client), không phù hợp với chat và notifications có xác nhận.
- Polling: Lãng phí tài nguyên, không realtime thực sự.

12. Testing Strategy

Layer	Tool	Target	Coverage Target
Unit (Backend)	Jest + ts-jest	Service, Repository	≥ 70%
Integration (Backend)	Supertest	Controllers (E2E)	Critical paths
Unit (Frontend)	Vitest + Testing Library	Packages (ui, shared)	≥ 60%
E2E	Playwright	User flows	10+ scenarios

Quyết định:

- **Backend:** Jest (NestJS ecosystem) + Supertest (HTTP testing).
- **Frontend packages:** Vitest (nhANH hơn Jest với esbuild).

- **E2E:** Playwright (cross-browser, auto-wait, network interception, tốt hơn Cypress cho testing complex flows).

Alternatives bị loại:

- Mocha/Chai: Thiếu assertion built-in, cần thêm thư viện, không phổ biến bằng Jest.
- Cypress: Chạy chậm hơn Playwright, không support Safari, không multi-tab.
- Vitest cho backend: NestJS Jest integration có sẵn, không muốn thay đổi.

13. CI/CD

Tiêu chí	GitHub Actions	GitLab CI	Jenkins	CircleCI
Hosting	✔️ GitHub	✔️ GitLab	Self-hosted	✔️ Cloud
Setup complexity	Thấp	Thấp	Cao	Thấp
Free tier	❤️ 2000 min/mo	400 min/mo	Unlimited (tự host)	6000 min/mo
Matrix builds	✔️	✔️	✔️	✔️
Caching	✔️	✔️	✔️	✔️
Docker build	✔️	✔️	✔️	✔️

Quyết định: GitHub Actions

- Repository đã trên GitHub, tích hợp sẵn, không cần third-party.
- Free tier 2000 minutes/tháng đủ cho project quy mô này.
- Matrix strategy cho phép test frontend + backend song song.

Pipelines:

- **Frontend:** lint → type-check → test → build
- **Backend:** lint → type-check → db:generate → prisma migrate deploy → test → test:e2e

14. AI Integration

Tiêu chí	OpenAI GPT-5.2	Gemini	Claude	Llama (self-hosted)
Vietnamese quality	❤️ Xuất sắc	Tốt	Tốt	Trung bình
API cost	❤️ Per-token	Rẻ hơn	Cao hơn	Free (self-hosted)
Structured output	✔️	❌	✔️	❌
Streaming	✔️	✔️	✔️	✔️
Setup complexity	Thấp (API key)	Thấp	Thấp	✔️ Cao (GPU)

Quyết định: OpenAI GPT (Model configurable qua env `OPENAI_MODEL`)

- Chất lượng tiếng Việt xuất sắc — quan trọng cho shopping concierge và support chatbot.

- Structured output (JSON mode) giúp parse kết quả dễ dàng cho các use case: product discovery, fit advisor, content generation.
- Model có thể thay đổi qua env variable (GPT-4o mini cho chat giá rẻ, GPT-5.2 cho phân tích nâng cao).

Use cases:

- Shopping concierge (gợi ý sản phẩm theo nhu cầu)
- Fit advisor (gợi ý size, phối đồ)
- Support chatbot (tự động trả lời FAQ, tạo ticket)
- Admin AI tools (phân tích doanh thu, gợi ý chiến lược)
- Content generation (mô tả sản phẩm, SEO)

15. Tổng kết Tech Stack

Layer	Technology	Version	Lý do chính
Monorepo	Turborepo + pnpm	^2.0.0 / ^9.0.0	Caching, task orchestration, strict mode
Frontend	Next.js + Module Federation	^15.0.0 / ^8.8.0	SSR, RSC, micro-frontend độc lập
Backend	NestJS + Fastify	^10.0.0 / ^4.28.0	Structure chuẩn, hiệu năng cao
Database	PostgreSQL + Prisma	16 / ^5.16.0	ACID, type safety, migration tự động
Cache & Queue	Redis + BullMQ	7 / ^5.71.1	Đa năng (cache + queue)
Search	MeiliSearch	latest	Dễ setup, hỗ trợ tiếng Việt
State	TanStack Query + Zustand	^5.50.0 / ^4.5.0	Server state + client state riêng biệt
UI	Tailwind CSS + Radix UI	^4.0.0 / ^1.x-2.x	Bundle nhỏ, headless, customizable
Real-time	Socket.IO	^4.7.0	Auto reconnect, room, fallback
Testing	Jest + Vitest + Playwright	^29.7.0 / ^1.0.0 / ^1.61.0	Đúng tool cho từng layer
CI/CD	GitHub Actions	—	Tích hợp sẵn, free
AI	OpenAI GPT	5.2	Chất lượng tiếng Việt, structured output
Payment	Stripe + PayPal + VNPAY + MoMo + ZaloPay	—	Phủ cả quốc tế và Việt Nam